Dated : 28/03/2026

AI-12 Python Assignment

Introduction to Python:

Source-W3Schools

Topic
Getting started with python
Python Syntax
Python Output
Python Comments
Python Variables
Python Data Types
Python Numbers
Python Casting
Python Strings
Python Booleans
Python Operators

Getting Started With Python

What is Python?

Python is a popular, high-level programming language created by Guido van Rossum and released in 1991. It is a versatile tool used across various industries for web development, software creation, complex mathematics, and system scripting. Because of its multi-purpose nature, it has become one of the most widely adopted languages in the world.

What Can Python Do?

Python acts as a powerful engine for building web applications on servers and managing intricate workflows alongside other software. It excels at connecting to database systems, handling big data, and performing high-level mathematical calculations. Whether you are performing rapid prototyping or building production-ready software, Python provides the tools to read, modify, and manage files efficiently.

Why Python?

Developers choose Python because it is cross-platform, running seamlessly on Windows, Mac, Linux, and even Raspberry Pi. Its simple, English-like syntax allows programmers to write logic in fewer lines than languages like Java or C++. Additionally, it runs on an interpreter system, meaning code is executed as soon as it is written, which significantly speeds up the development process.

Python Syntax vs. Other Languages

Python is uniquely designed for readability, using new lines to complete commands instead of the semicolons or parentheses found in other languages. A defining feature is its use of indentation (whitespace) to define scope, such as the body of a loop or function. This replaces the curly brackets used in other languages, resulting in a clean, uncluttered visual structure.

```
# The Python version installed : Python 3.14
print("Hello, World!") # Console will return the output "Hello,
World!"
```

```
Hello, World!
```

To check the python version

In console type the command "python --version"

```
# For the system use the following command

import sys # imports the sys system module

print(f"Python Version : {sys.version}") # print function with format
string to resturn the python version
```

```
Python Version : 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03)
[MSC v.1944 64 bit (AMD64)]
```

Console outputs

```
PS D:\TheDeuce\File_DS> python
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC
v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
Ctrl click to launch VS Code Native REPL
>>> print("Hello!, World!")
Hello!, World!
>>> exit()
```

Execute python script using terminal

```
PS D:\TheDeuce\File_DS> python -m class.assignments.hello
Hello!
PS D:\TheDeuce\File_DS>
```

Python Indentation

In Python, indentation refers to the specific number of spaces or tabs at the start of a line of code. Unlike other languages where indentation is used purely for visual clarity, Python uses it to define the structure and scope of the code, such as belonging to a loop or a function.

```
# Example code for the Python indentation
# This code will work because the indentation is correct
```

```
if 5>2:
    print("Five is greater than two!")
else:
    print("False statement")
```

```
Five is greater than two!
```

```
# Example code for the Python indentation
# This code will not work because the indentation is incorrect
if 5>2:
print("Five is greater than two!")
else:
print("False statement")
# Syntax Higlights also show the error in the code
```

Python Variables

In Python, a variable is essentially a named container or a label assigned to a specific location in a computer's memory used to store data. Unlike many other languages, you don't need to declare a variable's type explicitly; the variable is created the moment you first assign a value to it.

Variables can store different data types, such as integers, strings, or floating-point numbers, and their contents can be changed throughout the execution of a program. This flexibility makes managing and manipulating data intuitive, as you can refer to complex values by a simple, descriptive name.

```
integer_type = 10
string_type = "This is a string"
float_type = 3.14
print(f"Integer : {integer_type} \nString : {string_type} \nFloat :
{float_type}")
```

```
Integer : 10
String : This is a string
Float : 3.14
```

Comments

Comments are non-executable lines used to explain what the code is doing, making scripts much easier for humans to read and maintain. In Python, any text following the # symbol is ignored by the interpreter, allowing you to leave notes or "`comment out`" specific lines of code during testing.

They are essential for in-code documentation, helping other developers (or your future self) understand the logic behind complex sections. While Python doesn't have a specific syntax for multi-line comments, you can use a # for each line or utilize triple quotes `"""` as a workaround for longer explanations.

Different Comment types

- For a single line comments ''' or """ - Multi-line comments

Python Statements

- A computer program is a list of "instructions" to be "executed" by a computer.
- In a programming language, these programming instructions are called statements.
- The following statement prints the text "Python is fun!" to the screen:

```
print("Python is fun!")
```

Multiple Statements

Most of the programs contain multiple statements in a sequence. The Statements are executed one by one:

```
# Executed Line-by-Line Code
print("Hello World!")
print("Have a good day.")
print("Learning Python is fun!")
```

```
Hello World!
Have a good day.
Learning Python is fun!
```

Semicolons (Optional, Rarely Used)

Semicolons are optional in Python. You can write multiple statements on one line by separating them with ; but this is rarely used because it makes it hard to read:

```
# All the above code executed in single line, seperated by semicolons
print("Hello World!"); print("Have a good day."); print("Learning
Python is fun!")
```

```
Hello World!
Have a good day.
Learning Python is fun!
```

Python Outputs

```
# Print all the statements in a single line using end= parameter

print("This is the first print-statement.", end="")
print("This is the second print statement.")
print("This is the third print statement.", end="")
print("This is the fourth print statement.")
```

```
This is the first print-statement.This is the second print statement.
this is the third print statement.This is the fourth print statement.
```

```
# Print Numbers
print(3)
```

```
print(358)
print(50000)
```

```
3
358
50000
```

```
# Multiple types print statement

print("Integer", 35, "String", "This is a string", "Float", 3.14)

# Or
print(f"Integer: {25}\nString: {'This is a string'}\nFloat: {3.14}")
```

```
Integer 35 String This is a string Float 3.14
Integer: 25
String: This is a string
Float: 3.14
```

Python Comments

Comments in python is used for documentation of scripts, lines of code or a block of code.

- Comments can be used to explain Python code.
- Comments can be used to make the code more readable.
- Comments can be used to prevent execution when testing code.

Different Comment types

- # - For a single line comments
- ''' or """ - Multi-line comments

```
#print("Hello, World!")
print("Cheers, Mate!")
```

```
Cheers, Mate!
```

```
"""
This is a comment
written in
more than just one line
"""
print("Hello, World!")
```

```
Hello, World!
```

Variables

In Python, variables are symbolic names that act as references to values (objects) stored in memory. They are created the moment you first assign a value to them using the assignment operator (=) and do not require explicit type declaration, as Python is a dynamically typed language.

```
x = 4       # x is of type int
x = "Sally" # x is now of type str
print(x)
```

```
Sally
```

Casting

If you want to specify the data type of a variable, this can be done with casting. Casting also allows you to convert variable of existing type to a specific type.

```
# Casting the variables
int_var = 10
float_var = float(int_var) # Casting int to float
print("Float",float_var) # Expected output: 10.0
str_var = str(int_var) # Casting int to string
print("String",str_var) # Expected output: "10"
```

```
Float 10.0
String 10
```

Get the Type

Type casting in Python is the process of converting a value from one data type to another. This is useful for ensuring data compatibility, performing operations with the correct types, and handling user input.

There are two main types:

- implicit conversion.
- explicit conversion.

```
float_var = 10.0 # Declared a variable of type float
print(type(float_var)) # Expected output: <class 'float'>
```

```
<class 'float'>
```

Single vs Double Quotes

In Python, string variables can be declared using either single quotes (' ') or double quotes (" "); both methods are functionally identical and produce the same result.

```
variable_with_single_quotes = 'This is a string with single quotes'
variable_with_double_quotes = "This is a string with double quotes"
# Combining both types of quotes in a single print statement
print(f"{variable_with_single_quotes}\n{variable_with_double_quotes}")
```

```
This is a string with single quotes
This is a string with double quotes
```

Case-Sensitive

Python, the interpreter treats uppercase and lowercase letters as completely different characters. This means you can have multiple variables with the same letters, but different cases, and they will hold entirely separate values.

```
# Variable declared using lowercase
a = 20
A = "Character"
print(a) # Returns 20 an Integer
print(A) # Returns "Character" returns a String where the lowercase
and uppercase letters are treated as different variables
```

Variable Names

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)
- A variable name cannot be any of the Python keywords.

```
# All the valid variable names in Python
var = "John"
my_var = "John"
_var = "John" # can start with underscore
myVar = "John"
MYVAR = "John"
my_var2 = "John"

# Invalid variable names in Python
2var = "John" # Variable name cannot start with a number
my-var = "John" # Variable name cannot contain hyphens
my var = "John" # Variable name cannot contain spaces
```

```
  Cell In[24], line 2
    2var = "John" # Variable name cannot start with a number
    ^
SyntaxError: invalid decimal literal
```

Multi Words Variable Names

Variable names with more than one word can be difficult to read.

There are several techniques you can use to make them more readable: Camel Case : Each word, except the first, starts with a capital letter:

```
myVariableName = "John"
```

Pascal Case : Each word starts with a capital letter:

```
MyVariableName = "John"
```

Snake Case : Each word is separated by an underscore character:

```
my_variable_name = "John"
```

Many Values to Multiple Variables

Python allows you to assign values to multiple variables in one line:

The following script can be taken as an example for assigning values to a particular variable seprated by comma's.

```
# assigning multiple values to multiple variables in a single line
x, y, z = "Orange", "Banana", "Cherry"
print(x)
print(y)
print(z)
```

```
Orange
Banana
Cherry
```

One Value to Multiple Variables

And you can assign the same value to multiple variables in one line:

```
# assigning the same value to multiple variables in one line
x = y = z = "Orange"
print(x)
print(y)
print(z)
```

```
Orange
Orange
Orange
```

Unpack a Collection

If you have a collection of values in a list, tuple etc. Python allows you to extract the values into variables. This is called unpacking.

```
# Unpacking the list to vaiables
fruits = ["apple", "banana", "cherry", "orange", "grape"]
x, y, z = fruits
'''x will be assigned the value "apple",
```

```
y will be assigned the value "banana",
and z will be assigned the value "cherry"'''
print(x)
print(y)
print(z)
```

```
apple
banana
['cherry', 'orange', 'grape']
```

Output Variables

The print() function is the most common way to display variables in Python. Users can output a single variable or combine multiple variables and strings in one go.

```
# assigning a value to a variable and printing it
var = "Python is awesome"
print(var)
```

```
Python is awesome
```

```
x = "Python"
y = "is"
z = "awesome"
print(x, y, z) # Combining multiple variables in a single print
statement.
print(x + y + z) # This will print the combined string without spaces.
```

```
Python is awesome
Pythonisawesome
```

For numbers, the + character works as a mathematical operator:

Example

```
x = 5
y = 10
print(x + y)
```

Output : 15

in the print() function, when you try to combine a string and a number with the + operator, Python will give you an error:

Example

```
x = 5
y = "John"
print(x + y)
```

Output : TypeError: unsupported operand type(s) for +: 'int' and 'str'

The best way to output multiple variables in the print() function is to separate them with commas, which even support different data types:

Example

```
x = 5
y = "John"
print(x, y)
```

Output: 5 John

```
x= 5
y = "John"
print(x, y)
```

```
5 John
```

Global Variables

A global variable is defined outside of any function or class, making it accessible throughout the entire script or module.

```
# Create a variable outside of a function, and use it inside the
function

global_var = "I am a global variable"

def sample_func():
    print(global_var) # Accessing the global variable inside the
function

sample_func() # Calling the function to see the output of the global
variable
```

```
I am a global variable
```

```
# Create a variable inside a function, with the same name as the
global variable
x = "awesome"

def myfunc():
  x = "fantastic"
  print("Python is " + x)

myfunc()

print("Python is " + x)
```

```
Python is fantastic
Python is awesome
```

The `global` Keyword

Using the global keyword inside a function tells Python to treat the variable as if it were defined in the top-level scope of your script. This allows you to create or modify a variable that persists even after the function finishes running.

```
# If you use the global keyword, the variable belongs to the global
scope
def myfunc():
  global g
  g = "Awesome"

myfunc()

print("Python is " + g)
```

```
Python is Awesome
```

```
# To change the value of a global variable inside a function, refer to
the variable by using the global keyword:

x = "awesome" # This is a global variable

def myfunc():
  global x # Explicitly declaring that we are using the global
variable x
  x = "fantastic" # This will change the value of the global variable
x to "fantastic"

myfunc()

print("Python is " + x)
```

```
Python is fantastic
```

Python Data Types

Data types in Python are a way to classify data items. They represent the kind of value which determines what operations can be performed on that data. Since everything is an object in Python programming, Python data types are classes and variables are instances (objects) of these classes.

Basic Data types:

Type	Example
int	10
float	3.14
str	"hello" or 'hello'
bool	`True` / `False`

Type	Example
list	[1,2,3]
tuple	(1,2,3)
dict	{"name":"Alex"}

Getting the Data type

```
var = 50.5
print(type(var)) # Expected output: <class 'float'>
```

```
<class 'float'>
```

Types Definitions:

Data Type	Definition
str	Represents text (a sequence of characters) enclosed in quotes.
int	Represents whole numbers without decimals.
float	Represents numbers with decimal points.
complex	Represents numbers with real and imaginary parts (e.g., 1j).
list	An ordered, mutable collection of items.
tuple	An ordered, immutable collection of items.
range	Represents a sequence of numbers, often used in loops.
dict	Stores data in key-value pairs.
set	An unordered collection of unique items.
frozenset	An immutable version of a set.
bool	Represents True or False values.
bytes	Immutable sequence of bytes (binary data).
bytearray	Mutable sequence of bytes.
memoryview	Allows access to the internal data of an object without copying.
NoneType	Represents the absence of a value (None).

Data types Examples:

Example	Data Type
x = "Hello World"	str
x = 20	int
x = 20.5	float
x = 1j	complex
x = ["apple", "banana", "cherry"]	list

Example	Data Type
x = ("apple", "banana", "cherry")	tuple
x = range(6)	range
x = {"name" : "John", "age" : 36}	dict
x = {"apple", "banana", "cherry"}	set
x = frozenset({"apple", "banana", "cherry"})	frozenset
x = True	bool
x = b"Hello"	bytes
x = bytearray(5)	bytearray
x = memoryview(bytes(5))	memoryview
x = None	NoneType

Setting the Specific Data Type

If you want to specify the data type, you can use the following constructor functions:

Example	Data Type
x = str("Hello World")	str
x = int(20)	int
x = float(20.5)	float
x = complex(1j)	complex
x = list(("apple", "banana", "cherry"))	list
x = tuple(("apple", "banana", "cherry"))	tuple
x = range(6)	range
x = dict(name="John", age=36)	dict
x = set(("apple", "banana", "cherry"))	set
x = frozenset(("apple", "banana", "cherry"))	frozenset
x = bool(5)	bool
x = bytes(5)	bytes
x = bytearray(5)	bytearray
x = memoryview(bytes(5))	memoryview

```
# python bytes function()
a = bytes(1)
print(a)
```

```
#Python bytearray() function which returns a bytearray object which is
a mutable sequence of integers.
b = bytearray(5)
print(b)

# memoryview() function which only takes bytes as an argument and
returns a memory view object.
x = memoryview(bytes(5))
print(x)
print(type(x))
```

```
b'\x00'
bytearray(b'\x00\x00\x00\x00\x00')
<memory at 0x000001B789B49C00>
<class 'memoryview'>
```

Python Numbers

- int: Whole numbers without a decimal point (e.g., 5, -10, 1000).
- float: Numbers with a decimal point or in exponential form (e.g., 3.14, -0.5, 2.0e3).
- complex: Numbers with a real and imaginary part, written as a + bj (e.g., 2 + 3j).

```
# Integer is a whole number, positive or negative, without decimals,
of unlimited length.

x,y,z = 1, 243543544523, -100
print(f"{type(x)} : {x}\n{type(y)} : {y}\n{type(z)} : {z}")
```

```
<class 'int'> : 1
<class 'int'> : 243543544523
<class 'int'> : -100
```

```
# Float is a number, positive or negative, containing one or more
decimals.
# Float can also be scientific numbers with an "e" to indicate the
power of 10.
a,b,c = 1.5, -2.3, 3.14e-23
print(f"{type(a)} : {a}\n{type(b)} : {b}\n{type(c)} : {c}")
```

```
<class 'float'> : 1.5
<class 'float'> : -2.3
<class 'float'> : 3.14e-23
```

```
# Complex numbers are written with a "j" as the imaginary part
d,e,f = 2+3j, -1j, 5j
print(f"{type(d)} : {d}\n{type(e)} : {e}\n{type(f)} : {f}")
```

```
<class 'complex'> : (2+3j)
<class 'complex'> : (-0-1j)
<class 'complex'> : 5j
```

```
# Using all of the above types in a single code and perform type
conversion
```

```
x = 1    # int
y = 2.8  # float
z = 1j   # complex

#convert from int to float:
a = float(x)

#convert from float to int:
b = int(y)

#convert from int to complex:
c = complex(x)
# d = int(c)

print(a)
print(b)
print(c)
# print(d) # This will throw out an error because complex numbers
cannot be converted back to
print(type(a))
print(type(b))
print(type(c))
```

```
1.0
2
(1+0j)
<class 'float'>
<class 'int'>
<class 'complex'>
```

```
print(87.34e1)
```

```
873.4
```

Random Number

Python does not have a random() function to make a random number, but Python has a built-in module called random that can be used to make random numbers:

```
# import Random module
import random as rand # aliased Random function to rand
# Generate a random number between 2 and 50
print(rand.randrange(2,50))
```

```
31
```

Python Casting

Python Casting refers to the process of converting a variable from one data type to another using built-in functions. It is commonly performed using functions like int(), float(), str(), and bool() to ensure compatibility between different data types. Casting can be implicit

(automatic by the interpreter) or explicit (manually specified by the programmer). Explicit casting is often used to avoid type errors during operations. This mechanism is essential for data manipulation, input handling, and ensuring correct program behavior.

- int() - constructs an integer number from an integer literal, a float literal (by removing all decimals), or a string literal (providing the string represents a whole number)
- float() - constructs a float number from an integer literal, a float literal or a string literal (providing the string represents a float or an integer)
- str() - constructs a string from a wide variety of data types, including strings, integer literals and float literals

```
# Conversion to int
x = int(5.35)
y = int("2")
z = int(5)
print(f"{x}\n{y}\n{z}")
print(f"{type(x)}\n{type(y)}\n{type(z)}")

# Conversion to float
a = float(5)
b = float("3.14")
print(f"\n\n{a}\n{b}")
print(f"{type(a)}\n{type(b)}")

# Conversion to string
s1 = str(5)
s2 = str(3.14)
print(f"\n\n{s1}\n{s2}")
print(f"{type(s1)}\n{type(s2)}")

# Conversion to boolean
e = bool(0)
f = bool(1)
print(f"\n\n{e}\n{f}")
print(f"{type(e)}\n{type(f)}")
```

```
5
2
5
<class 'int'>
<class 'int'>
<class 'int'>


5.0
3.14
<class 'float'>
<class 'float'>
```

```
5
3.14
<class 'str'>
<class 'str'>


False
True
<class 'bool'>
<class 'bool'>
```

Python Strings

Python Strings are sequences of characters used to represent text data, enclosed within single `'`, double `"`, or triple quotes `'''` / `"""`. They are immutable, meaning their contents cannot be changed after creation. Strings support indexing, slicing, and a wide range of built-in methods such as `upper()`, `lower()`, and `replace()`. They also support operations like concatenation and repetition. Strings are a fundamental data type used for handling and processing textual information in programs.

Quotes Inside Quotes

You can use quotes inside a string, as long as they don't match the quotes surrounding the string:

Example

```
print("It's alright") # used escape char due to the prsence of single quote
print("He is called 'Johnny'") # used escape char due to the prsence of double quote
print('He is called "Johnny"')
```

```
It's alright
He is called 'Johnny'
He is called "Johnny"
```

Assign String to a Variable

Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

Example

```
a = "Hello"
print(a)
```

```
Hello
```

Multiline Strings

Using triple quotes of single or double quotes for representing a multi-line string.

```
# Using triple quotes with """ for multi-line strings
a = """Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua."""
print(a,"\n")

#using triple quotes with ''' for multi-line strings
a = '''Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.'''
print(a)
```

```
Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.

Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.
```

Strings and Arrays

- Like many other popular programming languages, strings in Python are arrays of unicode characters.
- However, Python does not have a character data type, a single character is simply a string with a length of 1.
- Square brackets can be used to access elements of the string.

```
# Example program to demonstrate the functionality of the string
var = "This is a string variable"
print(var[1]) # Prints the second character or char at index 1 similar
to an array
```

```
h
```

Looping through a String

By using the loops like `for` and `while` we can iterate through the string

```
var = "Apple"
for i in var:
    print(i) # This will print each character in the string on a new
```

```
line.
```

```
A
p
p
l
e
```

String Length

We can get the length of a string by using an in-built function which returns the length called len().

```
# This will print the length of the string "Apple" which is 5
print(len(var))
```

```
5
```

Check String

- To check if a certain phrase or character is present in a string, we can use the keyword in.
- To check if a certain phrase or character is NOT present in a string, we can use the keyword not in.

```
var = "Example check ops"
# returns a boolean value if the statement is true or false
print("ops" in var)
# since ops is present in the string var, it will return True

### Example for NOT in operator
print("help" not in var)
# since help is not present in the string var, it will return True
```

```
True
True
```

Check if NOT

To check if a certain phrase or character is NOT present in a string, we can use the keyword not in.

```
# taking example for NOT in operator
txt = "The best things in life are free!"
if "expensive" not in txt: # check if the word is not present
  print("No, 'expensive' is NOT present.")
else: # fallback if the word is present
  print("Yes, 'expensive' is present.")
```

```
No, 'expensive' is NOT present.
```

Slicing

You can return a range of characters by using the slice syntax.

Specify the start index and the end index, separated by a colon, to return a part of the string.

```
b = "Hello, World!"
print(b[2:5]) # Starts at index 2 and ends at index 4

# to slice from the start
print("Slicing from the start")
print(b[:5]) # Starts at the beginning and continues to index 4 (index
5 is not included)

# Slicing to the end
print("Slicing to the end")
print(b[2:5]) # Starts at index 2 and ends at index 4

# Negative indexing
print("Negative Indexing")
print(b[-5:-2]) # Starts at index -5 and ends at index -3 (index -2 is
not included)
```

```
llo
Slicing from the start
Hello
Slicing to the end
llo
Negative Indexing
orl
```

Python - Modify Strings

- **Upper Case** : Converts a string to upper case. use `str.upper()`
- **Lower Case** : Converts a string to lower case. use `str.lower()`
- **Remove Whitespace** : Whitespace is the space before and/or after the actual text, and very often you want to remove this space. use `str.strip()`
- **Replace** : The `var.replace()` method replaces a string with another string.
- **Split String** : The `var.split()` method returns a list where the text between the specified separator becomes the list items.

```
# Python modify strings operations
# 1. uppercase() method to convert a string to uppercase
var = "hello world"
print(var.upper()) # Expected output: "HELLO WORLD"
print(var.lower()) # Expected output: "hello world"
var_1 = "    hello world     "
print(var_1.strip()) # Expected output: "hello world"
print(var.replace("h", "j")) # Expected output: "jello world" replaces
```

```
all the occurrences of "h" with "j"
print(var.split()) # Expected output: ['hello', 'world'] splits the
string into a list of words
```

```
HELLO WORLD
hello world
hello world
jello world
['hello', 'world']
```

String concatenation

String Concatenation in Python is the process of merging two or more strings into a single sequence using the + operator. Because Python strings are **immutable**, this operation doesn't modify the originals; instead, it creates a brand-new string object in memory by copying the characters from the operands. To prevent a `TypeError`, all involved variables must be of the `str` data type, often requiring explicit type casting for numbers. While intuitive for simple tasks, joining a large volume of strings is more efficiently handled by the `.join()` method to optimize memory allocation.

```
first_name = "Ravi"
last_name = "Kumar"
c = first_name + last_name # Concatenating the first name and last
name without space in between
print(c)

# with whitespaces
first_name = "Ravi"
last_name = "Kumar"
c = first_name + " " + last_name # Concatenating the first name and
last name with a space in between
print(c)
```

Format Strings

In Python, string formatting provides a technical workaround for the language's strict type safety, which prevents the direct concatenation of strings and numeric types. Modern Python primarily utilizes f-strings (formatted string literals), where a prefix f and curly braces {} allow expressions to be evaluated and cast into the string at runtime. Alternatively, the .format() method uses placeholders to inject variables into a template, handling the necessary type conversion internally. These methods are preferred over manual casting because they improve code readability and allow for precise control over decimal precision and padding.

F-Strings

F-String was introduced in Python 3.6, and is now the preferred way of formatting strings.

To specify a string as an f-string, simply put an f in front of the string literal, and add curly brackets {} as placeholders for variables and other operations.

```
age = 36
txt = f"My name is Kumar, I am {age}" # Using f-string to format the string with the variable age
print(txt)
```

```
price = 59
txt = f"The price is {price} dollars" # Using f-string to format the string with the variable price
print(txt)
```

Escape Character

To insert characters that are illegal in a string, use an escape character.

An escape character is a backslash \ followed by the character you want to insert.

An example of an illegal character is a double quote inside a string that is surrounded by double quotes:

```
# the following code will throw an error due to no escape characters and prsence of same quote

txt = "We are the so-called "Vikings" from the north."
print(txt)
```

```
# The escape character allows you to use double quotes when you normally would not be allowed:

txt = "We are the so-called \"Vikings\" from the north."
print(txt)
```

Various Escape Characters

Code	Result
\'	Single Quote
\\	Backslash
\n	New Line
\r	Carriage Return
\t	Tab
\b	Backspace
\f	Form Feed
\ooo	Octal value
\xhh	Hex value

String Methods

Python has a set of built-in methods that you can use on strings.

Method	Description
capitalize()	Converts the first character to upper case
casefold()	Converts string into lower case
center()	Returns a centered string
count()	Returns the number of times a specified value occurs in a string
encode()	Returns an encoded version of the string
endswith()	Returns true if the string ends with the specified value
expandtabs()	Sets the tab size of the string
find()	Searches the string for a specified value and returns the position
format()	Formats specified values in a string
format_map()	Formats specified values in a string
index()	Searches the string for a specified value and returns the position
isalnum()	Returns True if all characters in the string are alphanumeric
isalpha()	Returns True if all characters in the string are in the alphabet
isascii()	Returns True if all characters in the string are ASCII characters
isdecimal()	Returns True if all characters in the string are decimals
isdigit()	Returns True if all characters in the string are digits
isidentifier()	Returns True if the string is an identifier
islower()	Returns True if all characters in the string are lower case
isnumeric()	Returns True if all characters in the string are numeric
isprintable()	Returns True if all characters in the string are printable
isspace()	Returns True if all characters in the string are whitespaces
istitle()	Returns True if the string follows title case rules
isupper()	Returns True if all characters in the string are upper case
join()	Joins the elements of an iterable to the end of the string
ljust()	Returns a left justified version of the string
lower()	Converts a string into lower case
lstrip()	Returns a left trim version of the string
maketrans()	Returns a translation table for translations
partition()	Returns a tuple where the string is split into three parts
replace()	Returns a string with a specified value replaced
rfind()	Searches the string and returns the last position found
rindex()	Searches the string and returns the last position found
rjust()	Returns a right justified version of the string
rpartition()	Returns a tuple where the string is split into three parts

Method	Description
rsplit()	Splits the string and returns a list
rstrip()	Returns a right trim version of the string
split()	Splits the string and returns a list
splitlines()	Splits the string at line breaks and returns a list
startswith()	Returns true if the string starts with the specified value
strip()	Returns a trimmed version of the string
swapcase()	Swaps cases (lower ↔ upper)
title()	Converts the first character of each word to upper case
translate()	Returns a translated string
upper()	Converts a string into upper case
zfill()	Pads the string with leading zeros

Python Booleans

Booleans represent one of two values: True or False.

Boolean Values

In programming you often need to know if an expression is True or False.

You can evaluate any expression in Python, and get one of two answers, True or False.

When you compare two values, the expression is evaluated and Python returns the Boolean answer:

```
a = 200
b = 33

if b > a:
  print("b is greater than a")
else:
  print("b is not greater than a")
```

Various Boolean Operations

```
print(bool("Hello")) # If the string is not empty, it will return True
print(bool(not(15))) # Returns the negation of the boolean value
```

```
True
False
```

```
# Non-empty strings and non-zero returns True
x = "Hello"
y = 15

print(bool(x)) # Returns True
print(bool(y)) # Returns True
```

```
True
True
```

```
# Non-empty strings and non-zero returns True
x= bool("abc") # Returns True
y=bool(123) # Returns True
z=bool(["apple", "cherry", "banana"]) # Returns True
print(x)
print(y)
print(z)
```

```
True
True
True
```

```
# All the following values are considered False in Python
a= bool(False)
b= bool(None)
c= bool(0)
d= bool("")
e= bool(())
f= bool([])
g= bool({})
print(f"{a}\n{b}\n{c}\n{d}\n{e}\n{f}\n{g}")
```

```
False
False
False
False
False
False
False
```

```
# Returns 0, as 0 is considered False in Python
class myclass():
  def __len__(self):
    return 0

myobj = myclass()
print(bool(myobj))
```

```
False
```

```
# If function returns True, the statement will return YES!
def myFunction() :
  return True

if myFunction():
  print("YES!")
else:
  print("NO!")
```

YES!

```
x = 200
print(isinstance(x, str))
```

```
False
```